

AI ASSISTED CODING

NAME: **KASTHURI SAI VARDHAN REDDY**

ROLL.NO:**2503A51266**

BATCH : **43**

ASSIGNMENT.NO : **3.1**

1. Question 1: Zero-Shot Prompting (Palindrome Number Program) Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling

PROMPT: create a simple python program to check whether given number is a palindrome

CODE:

```
num = int(input("Enter a number: "))

original = num
reverse = 0

while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num //= 10

if original == reverse:
    print(" the number is a Palindrome")
else:
    print("the number is not a Palindrome")
```

OUTPUT:

```
Enter a number: 12345
the number is not a Palindrome

Enter a number: 12321
the number is a Palindrome
```

```

Enter a number: 010
the number is not a Palindrome
Enter a number: -121
the number is not a Palindrome
Enter a number: ABC
Traceback (most recent call last):
  File "c:\Users\BHAVYASRI\OneDrive\Desktop\AISC LAB\is_palindrome.py", line 1,
in <module>
    num = int(input("Enter a number: "))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'ABC'

```

The function should be tested with normal, boundary, negative, and invalid inputs to ensure it works correctly in all cases.

2. Question 2: One-Shot Prompting (Factorial Calculation) Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity and correctness.

PROMPT: Given the example Input: 5 → Output: 120, generate a simple Python function to calculate the factorial of any given number.

Code:

```

def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print(factorial(5))

```

OUTPUT:120

TASK 1: Compare the generated code with a zero-shot solution.

Zero-shot solution:

The AI generates a factorial function without any example.

One-shot solution:

The AI uses the given example (5 → 120) to understand the expected input-output format.

Result:

The one-shot solution is more clearly aligned with the required output.

TASK 2: Examine improvements in clarity and correctness.

Clarity: Improved – the example makes the expected result clear.

Correctness: Improved – the AI can verify that `factorial(5)` should return 120.

Code structure: Simple and easy to understand.

3.Question 3: Few-Shot Prompting (Armstrong Number Check) Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

Task:

- Analyze how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.

PROMPT: Examples: Input: 153 → Output: Armstrong Number; Input: 370 → Output: Armstrong Number; Input: 123 → Output: Not an Armstrong Number. Using these examples, generate a simple Python function to check whether a given number is an Armstrong number. Also test the function with boundary values and invalid inputs.

CODE:

```
def is_armstrong(num):
    if not isinstance(num, int) or num < 0:
        return "Invalid Input"

    digits = str(num)
    power = len(digits)
    total = sum(int(digit) ** power for digit in digits)

    if total == num:
        return "Armstrong Number"
    else:
        return "Not an Armstrong Number"

# Test cases
test_cases = [153, 370, 123, 0, 9, -153, "abc"]

for num in test_cases:
    print(num, "→", is_armstrong(num))
```

OUTPUT:

```
153 → Armstrong Number
370 → Armstrong Number
123 → Not an Armstrong Number
0 → Armstrong Number
9 → Armstrong Number
-153 → Invalid Input
abc → Invalid Input
```

TASK1 : Analyze how multiple examples influence code structure and accuracy.

Multiple examples help the AI understand both positive and negative cases. The generated code is clearer and more accurate because it follows the given input-output pattern

TASK2: Test the function with boundary values and invalid inputs.

```
0 → Armstrong Number
9 → Armstrong Number
-153 → Invalid Input
abc → Invalid Input
```

The function correctly handles boundary values and invalid inputs.

4.Question 4: Context-Managed Prompting (Optimized Number Classification) Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.
- Compare the output with earlier prompting strategies

PROMPT: Write an optimized Python program that classifies an integer as Prime, Composite, or Neither. Requirements: validate the input and reject non-integer or invalid values; classify 0 and 1 as Neither; use an efficient method that checks divisibility only up to the square root of the number; provide clear, beginner-friendly code with comments; and test the program with multiple inputs.

CODE:

```
def classify_number(n):  
    # Input validation  
    if not isinstance(n, int):  
        return "Invalid Input"  
  
    # 0 and 1 are neither prime nor composite  
    if n <= 1:  
        return "Neither"  
  
    # Check divisibility only up to square root of n  
    i = 2  
    while i * i <= n:  
        if n % i == 0:  
            return "Composite"  
        i += 1  
  
    return "Prime"  
  
# Test cases  
numbers = [2, 7, 10, 15, 1, 0, -5, 17]  
  
for num in numbers:  
    print(num, "→", classify_number(num))
```

OUTPUT:

```
2 → Prime  
7 → Prime  
10 → Composite  
15 → Composite  
1 → Neither  
0 → Neither  
-5 → Neither  
17 → Prime
```

TASK1 : Ensure proper input validation.

```
The program checks whether the input is an integer.  
It correctly handles 0 and 1 as Neither.  
Negative numbers are also not classified as prime or composite.
```

TASK2: Optimize the logic for efficiency.

```
The program checks divisibility only up to  $\sqrt{n}$  instead of checking  
all numbers from 2 to  $n-1$ . Therefore, it is faster for large numbers.
```

TASK3: Compare the output with earlier prompting strategies

Prompting Strategy	Result
Zero-shot	Basic solution, fewer constraints
One-shot	Guided by one example
Few-shot	Guided by multiple examples
Context-managed	Clear requirements, validation, efficiency, and testing

Conclusion: Context-managed prompting produces a more controlled and efficient program because the AI receives clear instructions, constraints, validation requirements, and optimization goals.

5. Question 5: Zero-Shot Prompting (Perfect Number Check) Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

- Task:
- Record the AI-generated code.
 - Test the program with multiple inputs.
 - Identify any missing conditions or inefficiencies in the logic.

PROMPT:

Write a simple Python function to check whether a given number is a Perfect Number. The function should take an integer as input, calculate the sum of its proper divisors, and return "Perfect Number" if the sum equals the original number, otherwise return "Not a Perfect Number". Also test the function with multiple inputs.

CODE:

```
def is_perfect(n):
    if n <= 1:
        return "Not a Perfect Number"

    total = 0

    for i in range(1, n):
        if n % i == 0:
            total += i

    if total == n:
        return "Perfect Number"
    else:
        return "Not a Perfect Number"

# Test cases
numbers = [6, 28, 12, 1, 0, 496]

for num in numbers:
    print(num, "->", is_perfect(num))
```

OUTPUT:

```
6 -> Perfect Number
28 -> Perfect Number
12 -> Not a Perfect Number
1 -> Not a Perfect Number
0 -> Not a Perfect Number
496 -> Perfect Number
```

TASK 3: Identify any missing conditions or inefficiencies in the logic

Missing Conditions:

- The function does not validate non-integer inputs such as "abc".
- Negative numbers should be handled explicitly.

- Very large numbers may take a long time.

Inefficiency:

- The loop checks every number from 1 to $n-1$.
- A more efficient solution can check divisors only up to $\sqrt{n}$.

Conclusion:

The generated code is logically correct for normal positive integers, but input validation and divisor-checking optimization can be improved.

6. Question 6: Few-Shot Prompting (Even or Odd Classification with Validation) Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

- Input: 8 → Output: Even
- Input: 15 → Output: Odd
- Input: 0 → Output: Even Task:
- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs

PROMPT: Examples: Input: 8 → Output: Even; Input: 15 → Output: Odd; Input: 0 → Output: Even. Using these examples, generate a simple Python program to determine whether a given number is even or odd, with proper input validation for non-integer values. Also test it with negative numbers and invalid inputs.

CODE:

```
def check_even_odd(num):
    if not isinstance(num, int):
        return "Invalid Input"

    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

# Test cases
numbers = [8, 15, 0, -4, -7, "abc", 5.5]
```

```
for num in numbers:  
    print(num, "→", check_even_odd(num))
```

OUTPUT:

```
8 → Even  
15 → Odd  
0 → Even  
-4 → Even  
-7 → Odd  
abc → Invalid Input  
5.5 → Invalid Input
```

TASK 1: Analyze how examples improve input handling and output clarity

The multiple examples help the **AI** understand the expected output format for both even and odd numbers. The example 0 → Even also helps clarify boundary-case handling.

Input validation is included because the prompt explicitly asks the **AI** to handle non-integer inputs.

Conclusion: Few-shot examples improve output clarity and consistency, while the validation requirement ensures that invalid inputs are handled safely.